Relatorio de Auditoria Completa (Muuday App)

Data da analise: 2026-03-27
Escopo: seguranca, logica de negocio, confiabilidade, UI/UX, operacao, prontidao de producao.

Resumo executivo

- Build de producao compila com sucesso.
- Existem riscos criticos/altos em seguranca e logica de agendamento.
- Ha gap de conformidade/operacao (paginas legais, observabilidade, CI/lint, testes).
- O plano recomendado e corrigir primeiro risco de versao + autorizacao + integridade de dados de booking, depois endurecer API e politicas.

Achados priorizados (para executar primeiro)

P0 - Critico

1. Next.js desatualizado com vulnerabilidades criticas conhecidas

- Evidencia: `next` em `14.2.5` em package.json
- Evidencia adicional: `npm audit` reportou vulnerabilidade critica em `next` (inclui middleware bypass e outros advisories).
- Risco: exposicao direta da aplicacao em producao.
- Acao recomendada: atualizar para `next@14.2.35` (ou versao segura mais recente compativel), revalidar build e regressao de middleware/auth.

P1 - Alto

2. Logica de autorizacao incorreta para profissionais em acoes de booking

- Evidencia: comparacao `booking.professional_id !== user.id` em lib/actions/manage-booking.ts, 65, 108, 142
- Evidencia complementar: query de agenda para profissional usa `professional_id = user.id` em app/(app)/agenda/page.tsx\agenda\page.tsx:27)
- Contexto: schema usa `bookings.professional_id -> professionals.id` (nao `profiles.id`).
- Risco: profissionais nao conseguem confirmar/cancelar/concluir corretamente; possivel quebra total do fluxo.
- Acao recomendada: sempre resolver `professional.id` a partir de `user.id` antes de comparar/consultar bookings.

3. Tampering de preco/duracao/horario no createBooking

- Evidencia: valores vindos do cliente sao persistidos diretamente em lib/actions/booking.ts, 63, 65
- Risco: usuario pode manipular preco, duracao e gerar inconsistencias comerciais.
- Acao recomendada: ignorar `priceBrl` e `durationMinutes` de input e usar somente `professional.session_price_brl` e `professional.session_duration_minutes` do banco.

4. API de waitlist vulneravel a abuso e inconsistencia

- Evidencia: `upsert` sem rate-limit/captcha em app/api/waitlist/route.ts
- Evidencia: erro de DB e ignorado e email ainda e enviado em app/api/waitlist/route.ts, 34, com sucesso retornado em 38
- Evidencia: CORS `*` em app/api/waitlist/route.ts
- Risco: abuso de envio de email, spam relay, metricas falsas e perda silenciosa de dados.

- Acao recomendada: validar payload estritamente (schema), aplicar rate limit (IP + email), opcional captcha e bloquear envio de email quando persistencia falhar.

5. Persistencia da waitlist provavelmente bloqueada por RLS

- Evidencia: policy atual `USING (false)` em supabase-waitlist-migration.sql, 18
- Evidencia: cliente server usa anon key em lib/supabase/server.ts, nao service role.
- Risco: gravação pode falhar para publico anonimo enquanto API responde sucesso.
- Acao recomendada: criar endpoint waitlist com service role dedicado server-only (nunca exposto), ou criar policy de insert anon com validacoes fortes.

6. Exposicao de dados de perfil para todos

- Evidencia: policy publica em supabase-schema.sql e coluna de email em supabase-schema.sql
- Risco: privacidade (emails/atributos) e scraping.
- Acao recomendada: separar `public_profiles` de `profiles` sensivel e restringir select de `profiles` para dono/admin.

P2 - Medio

7. Integridade de reviews depende de input de cliente

- Evidencia: `user_id` e `professional_id` vindos de props no client em ReviewForm.tsx\avaliar[bookingId]\ReviewForm.tsx:48), 49\avaliar[bookingId]\ReviewForm.tsx:49)
- Evidencia de policy fraca para consistencia de booking/profissional: supabase-schema.sql
- Risco: inconsistencias e abuso de dados (mesmo com `user_id = auth.uid()`).
- Acao recomendada: inserir review via server action/SQL function que derive `user_id` do token e valide `booking_id -> professional_id` no servidor.

8. Busca usa filtro composto com interpolacao de input bruto

- Evidencia: app/(app)/buscar/page.tsx\buscar\page.tsx:26)
- Risco: erros de consulta, resultados inesperados e superficie para bypass de filtros.
- Acao recomendada: sanitizar input e dividir em filtros parametrizados seguros.

9. Conversao de timezone no booking e declarada como simplificada

- Evidencia: comentario explicito e funcao simplificada em components/booking/BookingForm.tsx, 90
- Risco: horarios errados em fronteiras DST/fuso, conflitos falsos/duplicados.
- Acao recomendada: usar `fromZonedTime` / `date-fns-tz` no servidor e padronizar persistencia em UTC.

10. Modelo de notificacoes desalinhado entre UI e migration

- Evidencia UI: app/(app)/configuracoes/page.tsx\configuracoes\page.tsx:8-11) e defaults [15-17]
- Evidencia migration: supabase-notifications-migration.sql
- Risco: preferencias parcialmente ignoradas/gravadas em chaves divergentes.
- Acao recomendada: definir contrato unico de `notification_preferences` e migrar dados existentes.

11. Lint/qualidade bloqueados por setup interativo

- Evidencia: `npm run lint` pede configuracao de ESLint (sem baseline versionado).
- Risco: queda de qualidade em PR e regressao silenciosa.
- Acao recomendada: versionar configuracao ESLint + script CI.

P3 - Baixo

12. Callback auth sem tratamento de erro explicito

- Evidencia: `exchangeCodeForSession` sem branch de erro em app/auth/callback/route.ts, com redirect direto 29
- Risco: UX ruim e diagnostico dificil em falhas OAuth.

- Acao recomendada: tratar erro e redirecionar com mensagem/codigo de falha.

13. Falta de headers de seguranca padrao

- Evidencia: next.config.js sem `headers()` para CSP/HSTS/X-Frame-Options/etc.
- Risco: hardening insuficiente.
- Acao recomendada: aplicar baseline de headers de seguranca.

14. Paginas de compliance/operacao ausentes

- Evidencia: nao foram encontrados endpoints para termos, privacidade, cookies, ajuda/suporte.
- Risco: compliance, confianca do usuario e bloqueio de ads/parcerias.
- Acao recomendada: criar paginas legais e linkar no footer e telas de cadastro/login.

Melhorias de UI/UX e produto

1. Unificar fluxo de onboarding

- Hoje existe cadastro multi-step, social login e completar-conta.
- Definir state machine unica: `registered -> profile_complete -> professional_profile_pending -> approved`.

2. Feedback de acao assicrona mais consistente

- Padronizar toasts, loading state e erros por dominio (auth, booking, profile).
- Evitar mensagens genericas para operacoes criticas.

3. Calendario e agenda

- Exibir timezone normalizado no card e no backend.
- Mostrar motivo de indisponibilidade (ocupado, fora da janela, antecedencia minima).

4. Admin UX

- Mover verificacao de admin para server component/layout e manter client apenas para interacao.
- Adicionar auditoria de acao administrativa (quem aprovou/rejeitou, quando).

5. Acessibilidade

- Rodar auditoria axe/lighthouse.
- Revisar contraste, foco visivel e labels de botoes icon-only.

Melhorias de engenharia/plataforma

1. CI minimo obrigatorio

- `typecheck`, `build`, `lint`, testes smoke.

2. Observabilidade

- Sentry (frontend + server route handlers + server actions).
- Logs estruturados com `request_id`.

3. Testes

- Unit: regras de booking, timezone, permissao.
- Integration: auth callback, waitlist API, CRUD de booking/review.
- E2E: login, cadastro, agendar, confirmar, concluir, avaliar.

4. Feature flags

- Social providers por ambiente.
- Waitlist email toggle para evitar envio indevido em staging.

Plano passo a passo para o Claude (ordem recomendada)

Fase 1 - Seguranca e integridade (bloqueante)

1. Atualizar dependencias de risco

- Atualizar `next` para versao segura da linha 14.2.x.
- Atualizar `@supabase/ssr` e revisar impacto.
- Rodar `npm audit` novamente e registrar diff.

2. Corrigir modelo de IDs de profissional em bookings

- Criar helper server-side `getProfessionalByUserId(user.id)`.
- Ajustar `agenda/page.tsx` para consultar por `professional.id`.
- Ajustar `manage-booking.ts` para comparar `booking.professional_id` com `professional.id`.

3. Blindar `createBooking`

- Remover `priceBrl` e `durationMinutes` do contrato de entrada publico.
- Buscar duracao/preco somente de `professionals`.
- Validar `scheduled_at` no servidor em UTC com timezone.

4. Corrigir waitlist endpoint

- Implementar validacao de payload (email, nome, tamanho, enum).
- Aplicar rate limit e opcional captcha.
- So enviar email se upsert/insert confirmar sucesso.
- Remover `Access-Control-Allow-Origin: *` e usar allowlist.

5. Corrigir RLS da waitlist e de profiles

- Waitlist: permitir insert controlado (ou usar service role server-only).
- Profiles: restringir SELECT de dados sensiveis.

Fase 2 - Confiabilidade e UX core

6. Revisar fluxo de review

- Mover insercao para server action.
- Validar que `booking_id` pertence ao usuario autenticado e profissional correto.

7. Normalizar timezone ponta a ponta

- Persistir UTC no backend.
- Exibir no fuso do usuario com funcoes unicas utilitarias.

8. Resolver desalinhamento de notificacoes

- Unificar chaves de JSONB entre migration e UI.
- Criar migration de backfill para dados existentes.

Fase 3 - Qualidade, compliance e operacao

9. Lint/CI

- Criar configuracao ESLint versionada.
- Adicionar workflow CI.

10. Compliance pages

- Criar `/termos`, `/privacidade`, `/cookies`, `/suporte`.
- Linkar no footer e telas de auth.

11. Observabilidade

- Integrar Sentry e logging estruturado.
- Adicionar dashboard de erros por rota.

Prompt pronto para usar com Claude (copiar e colar)

```
Quero executar o plano de hardening do Muuday em 3 fases.
Priorize seguranca e integridade de booking.

Fase 1:
1) Atualizar next/supabase para versoes seguras.
2) Corrigir mismatch professional_id vs user.id em agenda/manage-booking.
3) Blindar createBooking para ignorar preco/duracao vindos do cliente.
4) Reescrever waitlist API com validacao, rate-limit, CORS allowlist e envio de email apenas
com persistencia confirmada.
5) Ajustar RLS de waitlist/profiles para nao expor dados sensiveis.

Fase 2:
6) Mover criacao de review para server action com validacao de ownership.
7) Padronizar timezone (UTC no banco + exibicao por timezone do usuario).
8) Alinhar notification_preferences entre migration e UI.

Fase 3:
9) Configurar ESLint + CI.
10) Criar paginas legais (termos/privacidade/cookies/suporte).
11) Integrar observabilidade (Sentry + logs estruturados).

Para cada fase, entregue:
- patch de codigo
- migration SQL
- checklist de teste
- riscos remanescentes
```

Evidencias de execucao usadas nesta auditoria

- Build: `npm.cmd run build` (sucesso).
- Lint: `npm.cmd run lint` (bloqueado por setup interativo do ESLint).
- Security scan: `npm.cmd audit --json` (19 vulnerabilidades, incluindo 1 critica em dependencias).